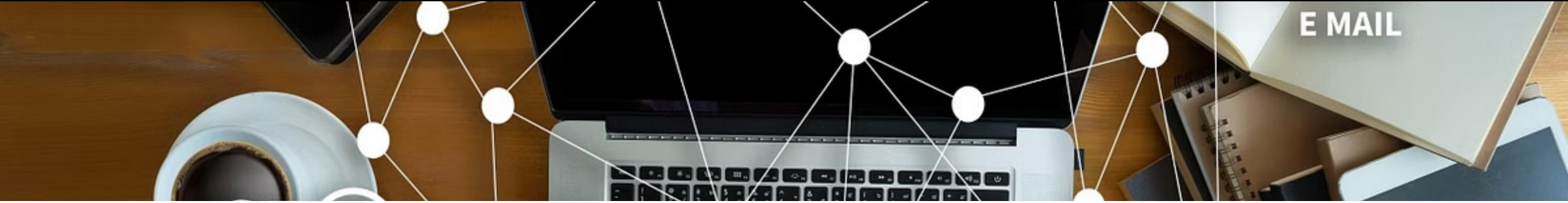
1SYX Case Study 01

When a diagnostic report is "correct on paper" but useless until you can challenge it

A diagnostic score is not the problem. The problem is what happens next. If the user cannot ask "why," push back, add context, and force a re-run, the report becomes a pretty PDF that nobody trusts, nobody acts on, and nobody defends in front of leadership. That is exactly what we fixed inside 1SYX.



The reality we built around

This is the actual moment your users hit.

A founder or a marketing lead opens Brand Diagnostics and sees the snapshot cards. Clarity is medium, Specificity is medium, GAP is not great, Persuasiveness is trending sideways. They can see the numbers, but they cannot trust the "why" yet because the tool might be judging them against the wrong segment, the wrong persona, the wrong geography, or the wrong competitor set.

So they do what humans do. They argue with the report. They say things like:

> "Our buyer is not SMB. We sell to enterprise."

> "We are not in Financial Services. We are in B2B software."

> "That competitor is not our competitor. We lose deals to a different one."

> "This post was written for India, not for the US."

Before this project, that argument lived outside the tool, in Slack, in a meeting, in a long message, and then everyone forgot the context and the report stayed the same. **We made that argument a first-class workflow inside 1SYX.**

The scenes we built for

These are the exact screens and behaviors from your UI that we designed the voice layer around.

Scene 1 — The user sees a score and wants to know what caused it

They open a metric like Clarity and click into "More Insight." They see: the score, the trend, the gap, the "what's going wrong" bullets, "your strengths" and "competitor strengths." But the real need is sharper. They want to say, out loud, in plain language: *"Show me the exact lines in my post that you're calling fragmented." "Why are you saying this is abstract? Point to the sentence." "Which part made you think this is for the wrong audience?"*

So we added a voice button inside the metric detail view, right next to the insight content, because this is where the questions happen, not in a separate chat screen. Their question is transcribed and pinned to that metric, not saved as a random note. The system replies with evidence tied to the same screen. This prevents the classic failure where a user feels "judged" but cannot see the basis, so they ignore the tool.

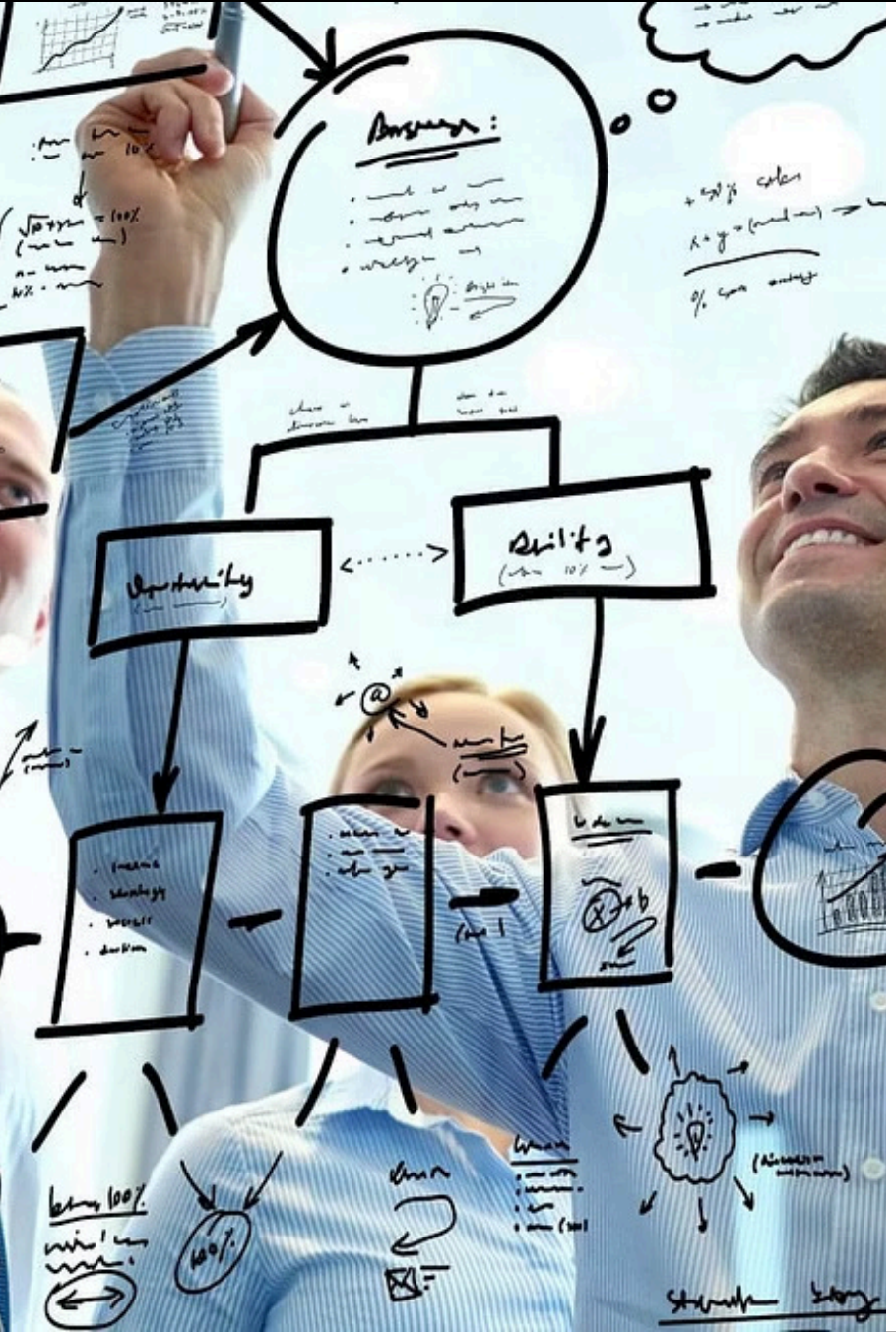
The scenes we built for

These are the exact screens and behaviors from your UI that we designed the voice layer around.

Scene 2 — The user changes the lens and wants the diagnostic to re-score

Your Diagnostics already have a Focus Lens selector, where the user can set industry, segment, persona, geography, and competitor context. That lens is not decorative. It changes what "good" looks like. So we built the voice interaction for the lens changes as well, because this is how people actually think: *"Switch this to Technology and Software, persona Head of Growth, geography UK." "Compare me against the strongest competitor only." "Run this as enterprise only, not SMB."*

We did not let voice changes silently re-run and overwrite anything, because users hate that. They want control. So voice changes create a **queued re-run, not an automatic re-run.** The system shows a simple confirmation step: This is the lens you had before / This is what you just changed / This is what will be recomputed. Then one button: Confirm re-run. If they do not confirm, nothing is recalculated. The report stays stable.



The scenes we built for

These are the exact screens and behaviors from your UI that we designed the voice layer around.

Scene 3 — The user wants actions, but only if the actions are based on the right diagnosis

Your Insights area already has a Gap Priority Map and a Unified Action Priority List with columns like urgency, effort, confidence, strategic edge, and gap score. The voice layer sits here for one reason. When the user says "tell me what to do," they do not want a generic to-do list. They want a to-do list tied to their chosen lens. So we enabled voice commands like: *"Show me only the immediate items with high impact and low effort." "Open the top item and tell me what a good example looks like." "Turn the top two into a plan for the next 7 days."*

When the user asks for a plan, the system writes back something practical, inside the product: It creates a structured task list tied to the specific gaps. It keeps the lens context attached so the plan does not become detached from the diagnosis.

What we delivered inside 1SYX

> This was not a new standalone assistant. It was a voice layer inside the screens that already exist.

Voice Diagnostic Dialogue Layer

Mic input inside existing diagnostic views

Context Ledger

Every spoken clarification stored and tied to its metric

Confirm-to-Rerun

No silent re-scoring — user controls every version

Written-Back Artefacts

Updated diagnostics with full traceability

DELIVERABLE 1

Voice Diagnostic Dialogue Layer

A mic input that appears inside:

- Brand Diagnostics snapshot and metric detail views
- "More Insight" modals
- Lens selector flows

Purpose: let the user challenge the logic at the exact point they disagree, not later.

DELIVERABLE 2

Context Ledger that the tool actually uses

Every spoken clarification is stored as:

1. **What the user said**
2. **Which metric or lens it was tied to**
3. **Whether it was accepted as a lens change, a user-supplied assumption, or a correction request**

> That context ledger is the difference between "a chat conversation" and "a diagnostic system that improves."

DELIVERABLE 3

Confirm-to-rerun mechanism

Voice input can propose changes, but it does not silently re-score. The user must confirm:

What changes	**What gets recomputed**
What new diagnostic version will be created	

DELIVERABLE 4

Written-back artefacts that make the work usable

After a confirmed rerun, 1SYX writes back:

Updated diagnostic version

With "what changed" attached

Updated priority map placement and action list ranking

Visible note on the metric cards

Showing which context was applied

This matters because **a diagnostic without traceability becomes a debate later.**

The safety rule that made it trustworthy

> The system is allowed to say "I can't accept that context automatically."

For example: If the user says "our buyer is enterprise," but the stored profile says "SMB," the tool does not pick a side quietly. It asks one tie-break question and then queues the re-run based on the chosen truth.

That prevents the **silent corruption problem**, where the diagnostic becomes inconsistent and nobody knows why.

What we learned

Users do not want a diagnostic that sounds confident.

They want a diagnostic that can be questioned without breaking.

Most disagreements are not about the score.

They are about the lens — meaning the wrong segment, persona, or competitor context.

A queued re-run with a clear confirmation step creates trust fast.

Because users feel they are driving the truth, not being judged by a black box.

Do your users trust your diagnostic?

- Do your users look at an insight, a score, or a recommendation in your product and immediately think, **"This is missing context,"** but they have no clean way to push back inside the tool? Do they end up arguing in Slack, adding disclaimers in meetings, or editing screenshots, because your product cannot capture their clarifications and re-run the analysis with that context?
- Do your best users stop trusting the system because they cannot ask "why," cannot challenge assumptions, and cannot see what changed when the inputs change?

If that sounds familiar, talk to us. We will take one high-stakes report or dashboard in your product, add a voice-first way for users to challenge and clarify in the moment, and add a confirmation step before any re-run changes the official output, so trust goes up and action happens inside the tool instead of outside it.

connect via neil.voishift@klyrr.com